

Лабораторна робота 1

Реалізація та перевірка зворотного поширення похибки

1 Мета та постановка задачі

Реалізовано прямий прохід і ручне зворотне поширення похибки у NumPy для мережі $4 \rightarrow 8 \rightarrow 3$. Градієнти перевірено через PyTorch autograd та центральні скінченні різниці. Усі перевірки правильної реалізації пройдено; навмисну помилку усереднення виявлено обома способами.

Досліди виконано на початкових параметрах і повній навчальній вибірці зі 105 об'єктів.

2 Підготовка даних

Використано Iris із `sklearn.datasets.load_iris()`: 150 об'єктів, 4 числові ознаки та 3 класи. Генератор `np.random.default_rng(0)` послідовно перемішує індекси класів 0, 1, 2. Перші 35 індексів кожного класу включено до train, решту 15 - до test. Розміри вибірок становлять 105 і 45 об'єктів відповідно.

Середні та стандартні відхилення ознак визначено лише на train із `ddof=0`. До test застосовано ті самі статистики. Для ознаки j :

$$\mu_j = \frac{\sum_i x_{ij}}{N}, \quad \sigma_j^2 = \frac{\sum_i (x_{ij} - \mu_j)^2}{N}$$
$$\tilde{x}_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

3 Архітектура та ініціалізація

Архітектура: $\text{Linear}(4,8) \rightarrow \text{ReLU} \rightarrow \text{Linear}(8,3)$. Вихідний шар повертає логіти. Для ініціалізації створено новий `default_rng(0)`, незалежний від поділу даних; $W1$ і $W2$ згенеровано саме в такому порядку.

Параметр	Форма	Ініціалізація
$W1$	4×8	He: $\text{Normal}(0, \text{std} = \sqrt{2/4})$
$b1$	8	Нулі
$W2$	8×3	Xavier: $\text{Normal}(0, \text{std} = \sqrt{2/11})$
$b2$	3	Нулі

Кількість параметрів: $32 + 8 + 24 + 3 = 67$. Форми ваг у NumPy — (in, out).

4 Прямий і зворотний проходи

Рядки X відповідають об'єктам. Зсуви додаються до кожного рядка за правилами broadcasting. Прямий прохід:

$$Z_1 = XW_1 + b_1, \quad A_1 = \text{ReLU}(Z_1), \quad Z_2 = A_1W_2 + b_2$$

Стабільну крос-ентропію обчислено через log-softmax зі зсувом на максимум кожного рядка:

$$m_i = \max_k Z_{2,ik}, \quad S_{ik} = Z_{2,ik} - m_i$$
$$\log P_{ik} = S_{ik} - \log(\sum_j \exp(S_{ij})), \quad L = -\frac{1}{N} \sum_i \log P_{i,y_i}$$

Після зсуву експоненти не перевищують 1. Втрата обчислюється безпосередньо з $\log P$, що дозволяє уникнути $\log(0)$ для ймовірностей, які округлилися до нуля.

Ручний backward

Y - one-hot матриця міток, D_2 та D_1 — похідні середньої втрати за передактиваціями відповідних шарів. Застосовано ланцюгове правило:

$$D_2 = \frac{P - Y}{N}$$
$$\nabla_{W_2} L = A_1^T D_2, \quad \nabla_{b_2} L = \sum_i D_{2,i}$$
$$D_1 = (D_2 W_2^T) \odot 1[Z_1 > 0]$$
$$\nabla_{W_1} L = X^T D_1, \quad \nabla_{b_1} L = \sum_i D_{1,i}$$

Множник $1/N$ враховано один раз у D_2 ; градієнти зсувів підсумовуються за об'єктами без повторного ділення. $\odot$ означає поелементне множення. Похідну ReLU в нулі прийнято рівною 0, як у PyTorch.

5 Перевірка правильної реалізації

Звірка з PyTorch

Створено еквівалентну мережу у float64. Параметри NumPy скопійовано до nn.Linear з транспонуванням W1 і W2, оскільки PyTorch зберігає ваги у формі (out, in). Після loss.backward() градієнти ваг транспоновано назад. Функція cross_entropy отримує логіти та reduction="mean".

Порівнюються всі 67 компонентів градієнта та втрата. Критерій: усі значення скінченні та $|a - b| \leq 10^{-12}$ для кожної пари. У таблиці наведено максимальну абсолютну різницю.

Втрата NumPy: 1.4562007801142816

Втрата PyTorch: 1.4562007801142818

Величина	Максимальна різниця	Пройдено
Loss	2.220446e-16	Так
Градiєнт W1	2.775558e-17	Так
Градiєнт b1	3.295975e-17	Так
Градiєнт W2	1.110223e-16	Так
Градiєнт b2	1.249001e-16	Так

Чисельне диференціювання

Для W1[0,0], b1[0], W2[0,0], b2[0] по черзі змінено один параметр на $\pm \epsilon$ від початкового значення, $\epsilon = 10^{-6}$. На кожному збуренні обчислено середню втрату на тих самих 105 об'єктах. Параметр гарантовано відновлюється у finally.

$$g_{num,i} = \frac{L_+ - L_-}{2\epsilon}, \quad |g_{num,i} - g_{manual,i}| \leq 10^{-7}$$

Параметр	Ручний градієнт	Чисельна похідна	Абс різниця	Пройдено
W1[0,0]	-1.992018433e-02	-1.992018439e-02	6.034e-11	Так
b1[0]	-7.056611158e-02	-7.056611162e-02	3.633e-11	Так
W2[0,0]	1.352302086e-01	1.352302087e-01	9.218e-11	Так
b2[0]	8.552164484e-02	8.552164499e-02	1.460e-10	Так

Усі значення скінченні. Усі чотири перевірки пройдено. ReLU-маски для початкових і збурених параметрів збіглися, отже збурення не перетнули злам активації. Після перевірок параметри точно збігаються з початковими.

Максимальна розбіжність градієнтів із PyTorch становить приблизно $1.25 \cdot 10^{-16}$, а максимальна похибка чисельної перевірки — $1.46 \cdot 10^{-10}$. Обидва значення менші за встановлені допуски.

6 Дослід із навмисною помилкою

Прогноз до запуску

Якщо замінити $D2 = (P - Y)/105$ на $D2 = P - Y$, втрата залишиться незмінною: forward і параметри не змінюються. Усі ненульові градієнти збільшаться за модулем у 105 разів зі збереженням знака; нульові залишаться нульовими. Звірка втрати пройде, але перевірки ненульових градієнтів з PyTorch та центральними різницями мають виявити помилку.

Умови та результати

Помилковий режим вмикається аргументом --bug. Використано початкові ваги та ті самі 105 об'єктів. PyTorch-еталон, функцію середньої втрати й формулу чисельної похідної не змінено; оновлення параметрів не виконувалися.

Втрата NumPy: 1.4562007801142816

Втрата PyTorch: 1.4562007801142818

Величина	Максимальна різниця	Пройдено
Loss	2.220446e-16	Так
Градiєнт W1	1.851814e+01	Hi
Градiєнт b1	1.564109e+01	Hi
Градiєнт W2	3.419343e+01	Hi
Градiєнт b2	1.119709e+01	Hi

Параметр	Помилковий градієнт	Чисельна похідна	Абс різниця	Пройдено
W1[0,0]	-2.091619355e+00	-1.992018439e-02	2.072e+00	Hi
b1[0]	-7.409441716e+00	-7.056611162e-02	7.339e+00	Hi
W2[0,0]	1.419917190e+01	1.352302087e-01	1.406e+01	Hi
b2[0]	8.979772708e+00	8.552164499e-02	8.894e+00	Hi

Прогноз підтверджено. Втрата не змінилася, усі чотири групи градієнтів не пройшли звірку з PyTorch, усі чотири чисельні перевірки також виявили розбіжність. Відношення ненульових градієнтів дорівнює 105 з похибкою округлення; найбільший залишок не перевищує $7.11 \cdot 10^{-15}$.

7 Відтворення результатів

Потрібні uv та Python 3.12.

```
uv sync --locked
uv run --locked python main.py
uv run --locked python main.py --bug
```

Код виходу 0 означає очікуваний результат вибраного режиму, 1 — непередбачену невідповідність. Для --bug успіх означає виявлення помилкових градієнтів.

Контрольне середовище: Python 3.12.14; NumPy 2.2.5; PyTorch 2.7.0+cpu; scikit-learn 1.6.1; Windows, CPU. Прямі версії задано в pyproject.toml, усі транзитивні залежності та хеші - в uv.lock.

8 Висновки

Реалізовано ручний backpropagation для двошарової мережі з ReLU та стабільною крос-ентропією. Всі 67 компонентів градієнта узгоджуються з PyTorch у межах 10^{-12} ; чотири вибрані компоненти узгоджуються з чисельними похідними у межах 10^{-7} . Дослід показав, що правильне значення втрати саме по собі не гарантує правильності backward.

9 Джерела та інструменти

1. ЛР1 завдання.
2. Лекція 1.
3. Лекція 2.
4. Лекція 3.
5. OpenAI Codex використано для підготовки коду і звіту.